Reducing Notification Fatigue using a Portable Adaptive-Baseline PM2.5 Monitor for Persistently Polluted Cities

Abstract

Air pollution is a serious concern, particularly in cities like Dhaka where the air is almost never clean. Monitors with a fixed limit send too many alerts and users stop paying attention, which defeats the purpose. A portable system was built with an ESP32-C3, a PMS7003 and a BME280, soldered onto a perfboard and powered by one 18650 battery. The firmware keeps a sliding history of recent PM2.5 readings and only alerts on a statistically significant spike above the baseline. Alerts go to a smartphone over Bluetooth Low Energy. A React Native app and a Firebase backend were made to show live data and log spike events. One of the authors carried the device for two weeks in Dhaka, the first week in fixed-threshold mode and the second in spike-detection mode. The fixed-threshold mode produced dozens of alerts per day while spike-based mode only generated a few. The spike alerts were found to be considerably more meaningful, suggesting adaptive alerting can reduce alert desensitisation even in a heavily polluted city.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile devices**; *Interaction design*; *Empirical studies in HCI*; • **Applied computing** → **Environmental sciences**.

Keywords

Air quality monitoring, PM2.5, portable device, alert fatigue, ESP32, Dhaka

ACM Reference Format:

. 2026. Reducing Notification Fatigue using a Portable Adaptive-Baseline PM2.5 Monitor for Persistently Polluted Cities. In *Proceedings of 4th International Conference on Computing Advancements (ICCA 2026).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Dhaka is often considered one of the most polluted cities in the world. Daily $PM_{2.5}$ concentrations reach 150–200 $\mu g\ m^{-3}$ in winter, far above the WHO 24-hour guideline of 15 $\mu g\ m^{-3}$ [15]. It cannot be understated how serious the health consequences of long-term exposure are [9].

Most air-quality monitors notify the user whenever $PM_{2.5}$ crosses a fixed limit. In a city where the air is almost never below that limit, this means alerts all day. After a while people stop reading them. This is notification fatigue. The problem is that a monitor constantly saying the air is hazardous gives no new information. What actually matters is whether the air has gotten worse than what the person was already breathing. A jump from 150 to 190 can point to something nearby, like a construction site or a diesel bus idling at the kerb. That is information a user can act on. A monitor that cannot make that distinction is not very useful.

A portable monitor was built that maintains a sliding window of recent $PM_{2.5}$ readings and only alerts when the current value spikes well above that personal baseline. The hardware was kept small and battery-powered so it could be carried during a normal commute.

The key contributions of this work are as follows:

(1) A portable air quality monitor designed for environments where background $PM_{2.5}$ is persistently high.
(2) A context-aware alert mechanism based on an adaptive personal baseline, which considerably reduces unnecessary notifications.
(3) A low-cost hardware build on a perfboard using an ESP32-C3, a PMS7003, and a BME280, powered by a single 18650 cell.
(4) A two-week field study comparing fixed-threshold and spike-detection alerting for a single user commuting in Dhaka.

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 describes the hardware. Section 4 covers the firmware and software. Section 5 presents the field experience. Section 6 discusses limitations, and Section 7 concludes.

2 Literature Review

2.1 Sensor Viability

This was a suitable option for the project because of previous research done on the PMS7003. Comparing with reference instruments, Badura *et al.* reported $R^2 > 0.9$ for the $PM_{2.5}$ trends, but with high humidity, it over-estimates the trends. A follow-up study by the same team showed that it could be used for monitoring trends with regular calibration without causing issues [3]. Alfano *et al.*[1] summarized the existing low-cost PM sensors and discussed the still ongoing issues of calibration and unit-to-unit variability. The accuracy limitations are not so significant here since the goal is relative spike detection, not reporting absolute concentrations.

2.2 Portable Monitors with User Feedback

Building a portable monitor that sends readings to a phone is not a new idea. Most published work stops at reporting accuracy. Not many consider whether the alert logic actually works for the environment the device is used in.

Alvear-Puertas *et al.* mounted MQ-series sensors and an ESP32 on a vehicle and ran ML classification server-side over MQTT [2]. The system performed well in tests. However it was fixed to a vehicle and the alerts were static threshold-based. Suriano's SentinAir had a display and WiFi for remote control but no smartphone app and no adaptive alerting [14]. Sharma and Kumar connected an

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCA 2026, Dhaka, Bangladesh

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ESP32 to Blynk for phone visualisation, but notifications were still fixed-limit [13]. Haque *et al.* paired a PMS7003 with a web dashboard targeting developing countries [7]. Alerting was threshold-based. Palomeque-Mangut *et al.* used BLE to a wearable and got 97 % accuracy in controlled tests [11]. Nguyen *et al.* displayed $PM_{2.5}$ and CO_2 on a smartwatch with push alerts, but the triggers were fixed thresholds [10].

Data reaching a phone is not the gap. None of these systems changed what counts as an alert based on what the user had already been breathing. When the baseline never drops, that distinction matters a lot.

2.3 Alert Fatigue and the HCI Opportunity

In this portion of the review, alert fatigue research and its relevance to air quality monitoring will be highlighted. Alert fatigue has been studied in public health settings, though not in this domain. Baseman *et al.* showed that frequent un-prioritised notifications cause desensitisation over time [4]. Mehrotra *et al.* found that context-aware notification preferences reduce interruptions without losing important alerts [8]. However these ideas have not been applied to air quality monitors. However static thresholds remain the norm even in cities where the threshold is crossed the moment a user steps outside.

Portable sensor studies are also relevant here. Becker *et al.* reviewed 33 personal exposure feedback studies and found behaviour change was mixed, with low self-efficacy reported frequently [5]. Robinson *et al.* gave nineteen participants portable monitors and ran focus groups. Data gaps and a confusing air-quality index were the main barriers to continued use [12]. Bernasconi *et al.* had 16 participants wear a PM monitor for a walk in Milan. Awareness went up but a sense of powerlessness also increased because readings kept changing without any clear way to act [6]. Alerts that only appear when the user can actually do something work better than constant ones. This is an actual gap that must be filled. There is no previous system that integrates adaptive alerting with portable air quality hardware for environments that are persistently polluted.

2.4 Comparison of Existing Portable Systems

Table 1 showcases a summary of related portable systems. It takes into account the portability, feedback channel, alert logic as well as major drawbacks of the systems. This work has been located in the last row.

As can be seen, no previous system is a combination of pocket size hardware, phone based feedback and adaptive alerting based on the user's own recent exposure. In this paper, the Adaptive Baseline Spike Detection (ABSD) algorithm is presented to tackle this.

3 System Architecture and Interaction Design

Small and portable was the main requirement. The user needs a clear alert when something changes and a way to look back at events later.

3.1 Hardware Components and Physical Form

An ESP32-C3 SuperMini is at the centre. It is compact and has BLE built in, so no extra wireless module was needed. A PMS7003 laser particle counter processes the PM readings and a GY-BME280 breakout processes the temperature, humidity and pressure. Everything was soldered to a 5cm x 7cm double sided perfboard, wires were kept short. Figure 1 shows the internals while the system was running. Both the ESP32-C3 and the MT3608 boost converter have red LEDs that are clearly on. JST connectors were used so parts can be disconnected without cutting wires.

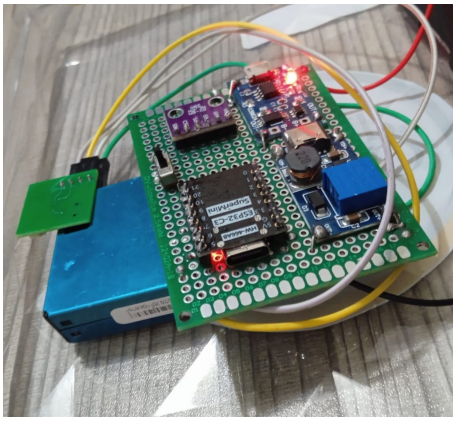


Figure 1: Internal hardware of the portable monitor. The large blue component is the PMS7003 sensor. The ESP32-C3 board (centre-bottom), the BME280 sensor (small purple board, upper-middle left), and the MT3608 boost converter (blue module with inductor, right) are all visible.

3.2 Power Architecture

It is powered by a single 3.7 V 18650 lithium-ion cell. The charging, overcharging and overdischarging protection is provided by a TP4056 module. The master power control is a slide switch.

Once the switch is on, power splits into two paths. Path A goes through an MT3608 boost converter set to 5 V. This rail only powers the PMS7003 because the sensor needs a stable 5 V for its internal fan and laser. Without it the fan can run slow and readings become unreliable. Path B goes directly into the RAW pin of the ESP32-C3, which has its own regulator producing 3.3 V. The BME280 is powered by that 3.3 V line. All grounds are tied together because both UART and I2C need a common reference. Without a common ground readings get noisy and the microcontroller can reset. All the parts are connected together as shown in Figure 2.

3.3 Physical Button for Baseline Reset

A pull-up resistor on GPIO10 was used for a tactile button to reset the baseline. A typical example of this is when someone leaves an air-conditioned building with a PM2.5 of 20 and steps into a street where it is 170. If the reset is not there, the old low readings stay in the buffer and the new high values look like a spike when they are not. When the button is pressed, the buffer is cleared and the device waits until it has enough new readings before alerting again. This was done without having to take out the phone. A physical button made that possible.

Table 1: Comparison of portable air-quality monitors that provide real-time user feedback.

Ref.	Year	Approach	Hardware & Method	User Feedback	Key Findings	Limitations
[2]	2022	Vehicle-mounted portable IoT	MQ sensors, ESP32, MQTT, cloud SVM	Cloud dashboard	ML computation feasible, >90% accuracy	Fixed to vehicle, threshold-based alerts
[14]	2021	Portable field unit (SentinAir)	Battery, integrated display, WiFi, MOx sensors	On-screen display	Real-world sensor inter-comparison	No smartphone app, no adaptive alerting
[13]	2020	IoT + Blynk server	ESP32, various sensors, Blynk cloud	Mobile app (Blynk)	Low-cost remote visualisation	Relies on fixed pollutant limits
[7]	2020	Low-cost IoT for developing countries	ESP32, PMS7003, web dashboard	Web interface	Feasible for remote monitoring	Fixed thresholds, not pocket-sized
[11]	2022	Wearable BLE monitor	MOS + PM sensor, BLE, smartphone app	Mobile app	97% lab / 82.3% field accuracy	Threshold-based alerts, no adaptive logic
[10]	2024	Smartwatch air quality monitor	Apple Watch, $PM_{2.5}/CO_2$ sensors, mobile app	Watch display, push alerts	Glanceable data raised awareness	Fixed notification triggers, small user sample
This work	2026	Portable spike detection	ESP32-C3, PMS7003, BME280, BLE, Firebase, React Native	Mobile app, web dashboard	Spike mode produced only a few alerts/day; fixed mode produced dozens	Single-user, two-week test, no humidity correction

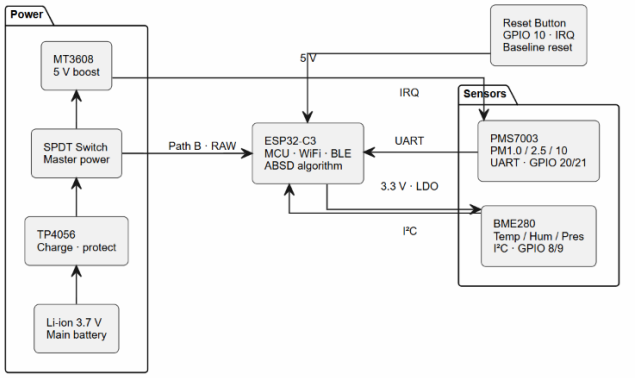


Figure 2: Hardware block diagram showing power delivery, sensor connections, and the processing unit.

3.4 Cost and Replicability

All components were bought from local shops in Dhaka. Table 2 shows the parts and prices in Bangladeshi Taka (BDT). Total cost was about 4,500 BDT, roughly 37 USD. Commercial portable monitors with similar sensors usually cost several hundred dollars. This design needs no custom PCB and can be put together with basic soldering skills.

4 Software Design and User Experience

Software has three parts: firmware on the ESP32-C3, a mobile app, and a cloud backend. Figure 3 shows how they connect.

4.1 Firmware and Spike Detection Algorithm

C++ with the Arduino framework was used for the firmware. The PMS7003 is read every 10 seconds over UART at 9600 baud and the BME280 is read over I2C with the Adafruit library. Each cycle measures temperature, humidity, $PM_{2.5}$ and PM_{10}.

ABSD algorithm is the core. The last 60 $PM_{2.5}$ values are stored in a circular buffer in memory, covering about 10 minutes at the 10-second sampling rate. The rolling mean and standard deviation are updated on each new reading. The standard deviation is floored at 5 $\mu g/m^3$. This flooring was not originally planned. During testing one afternoon the air was unusually stable and the standard deviation dropped so low that even a 2–3 $\mu g/m^3$ wobble triggered an alert. The phone buzzed every few minutes even though nothing had changed. Flooring the value at 5 fixed this problem.

Table 2: Bill of materials for the final device (prices in BDT).

Component	Qty	Cost (BDT)
PMS7003 dust sensor	1	2,590
ESP32-C3 SuperMini	1	539
BME280 sensor	1	420
TP4056 charger module	1	32
MT3608 boost converter	1	70
18650 Li-ion battery	1	420
18650 battery holder	1	25
5×7 cm veroboard	1	55
SPDT slide switch	1	5
JST connector	1	20
Capacitors (100 μF, 10 μF)	2	13
Wires, solder, glue, enclosure	–	300
Total		**4,489**

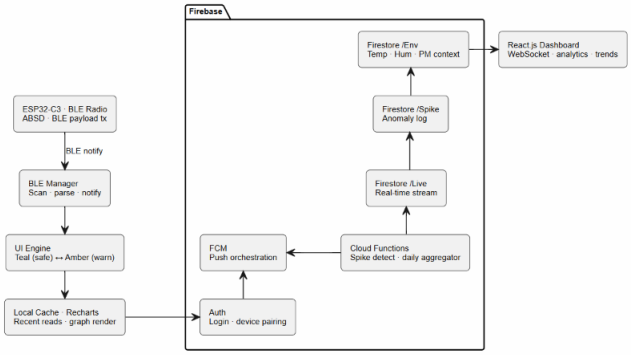


Figure 3: Software and cloud ecosystem: BLE, mobile app, Firebase, and web dashboard.

For a spike to be declared, the reading must exceed the mean by more than two standard deviations, the absolute rise should be more than 15 $\mu g/m^3$, and the reading must be between 1 and 1000 $\mu g/m^3$, since going above that value is generally a sign of faulty sensor. The 15 $\mu g/m^3$ cutoff was not taken from any reference. It came from watching the serial output for a few days and noticing natural drift rarely exceeded 10 $\mu g/m^3$ without an obvious nearby source. The validity range sounds unnecessary. However it caught a real problem during testing when a loose wire caused the sensor to report exactly zero for several minutes. Those zero readings would have corrupted the buffer.

A spike is only confirmed if at least three of the last five readings all satisfy the conditions. Once confirmed, a cooldown of around 10 minutes begins and no further alerts fire during that time. This stops the device from sending too many notifications during a long episode like slow traffic.

When a spike is confirmed and outside the cooldown window, the ESP32 writes the current sensor values to a BLE characteristic, which then triggers the notification on the phone. The reset button is a hardware interrupt, pressing it clears the buffer and resets the running sums Moreover, the device then enters a warm-up period until enough readings are collected.

4.2 Mobile Application

React Native and Expo were used so the same code runs on Android and iOS. BLE uses the react-native-ble-plx library. On first launch a connect button appears. Once paired it is replaced by a small status indicator.

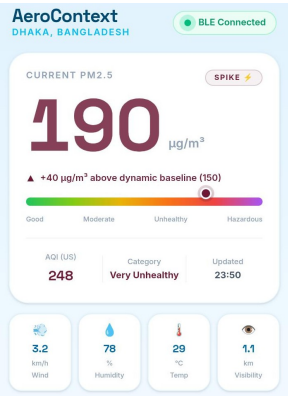


Figure 4: Main dashboard of the AeroContext app during a spike.

The main dashboard is shown in Figure 4. $PM_{2.5}$ is displayed in large text at the top. Normal background is teal. When a spike arrives the background turns amber and a short message appears below the number, for example "+40 $\mu g/m^3$ above baseline." A colour bar showing the AQI range from green to maroon is also shown. Below that, a line chart built with React Native Skia and Reanimated displays $PM_{2.5}$ alongside the running baseline, animating at 60 frames per second. Small cards lower down show temperature, humidity, wind speed and visibility. Humidity is shown because the PMS7003 reads higher than the true concentration in humid conditions.

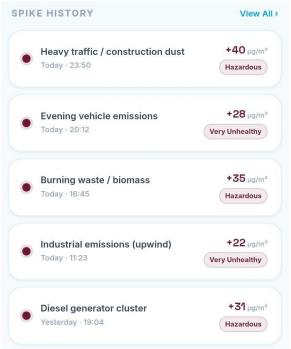


Figure 5: Spike history screen with logged events.

Two more screens let the user look back at past events. The spike history screen (Figure 5) lists every spike with a timestamp and $PM_{2.5}$ rise. The alert detail screen (Figure 6) expands a single event with the numbers, a short explanation, and buttons like "View Details" and "Avoid Route." The main screen has a recalibration button that sends a BLE write with the same effect as pressing the physical button.

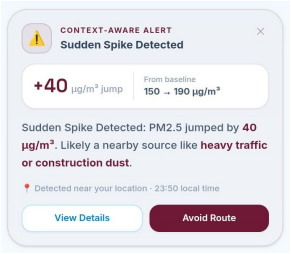


Figure 6: Context-aware alert card for a single spike.

Every reading and spike event syncs to Firebase when the phone has internet. $PM_{2.5}$, PM_{10}, temperature, humidity, and a record of each spike with its timestamp and magnitude are all stored.

4.3 Cloud Backend

Firebase Cloud Firestore was the database. Three collections per user: live readings, spike history, and full environmental logs. Two Cloud Functions were set up. One sends a push notification through Firebase Cloud Messaging when a spike is logged, so the alert reaches the user even if the app is closed. The other runs daily and calculates averages from the raw data.

4.4 Development and Debugging

There were a string of problems during the development. The BME280 kept failing to initialise, printing "BME280 NOT FOUND" to the serial monitor regardless of what was tried. The I2C address was changed, the wiring was checked multiple times. Eventually a loose connection on the SDA line turned out to be the problem. Tightening it fixed everything.

The PMS7003 required some work as well. The sensor sends 32-byte binary packets and finding the correct byte offsets for $PM_{2.5}$ and PM_{10} required trial and error.

The biggest problem was with Blynk. Blynk was used as a demo and to check the cloud features of the micro-controller as well. Initially, the dashboard showed every value as 1. The code was checked, the virtual pins were checked, the library version was checked. Nothing helped. After a few hours it was discovered that the datastream maximum values in the Blynk console were all set to 1 by default. Blynk was clamping every reading to that maximum no matter what the sensor reported. Changing the maximums fixed the dashboard instantly. That was a frustrating lesson and it is why the final version uses Firebase. Another reason being, Blynk was not as reliable as it was required.

5 Field Experience over Two Weeks

One of the authors used the device for two weeks while commuting in Dhaka during a regular summer. The prototype was placed in the side pocket of a backpack so air could flow in freely, which was required for the PMS7003 sensor. Fixed-threshold mode was used in week one, alerting every time $PM_{2.5}$ exceeded 150 $\mu g/m^3$, while for week two, spike-detection mode was used, and the baseline was reset each morning.

The first week proved this study correct. It was a tiring week. As soon as the building was left, alerts started and did not stop. There were three or four notifications per bus ride, even more during a walk across campus and another couple of times at lunch outside and on the way home. During lectures the constant buzzing was too distracting so the notification was silenced. After a while the notifications were not being read at all. They were just swiped away as it because part of the user's habit.

Unexpectedly, a heavy rainstorm mid-week dropped $PM_{2.5}$ considerably. However, the moment it stopped the numbers climbed again and the alerts came back in a burst, perhaps due to the resurfacing of pollutants after the downpour. By the end of the week the device was just a source of irritation.

Week two was completely different. The phone stayed quiet for long stretches even when the dashboard showed high $PM_{2.5}$. No alerts meant nothing had suddenly changed. That silence was useful. The first real spike came on the third day while walking past a construction site near Farmgate. $PM_{2.5}$ jumped from around 160 to 215 in under a minute. The phone buzzed once. The alert showed "+55 $\mu g/m^3$ above baseline." A dust cloud was clearly visible. The response was to cross to the other side of the road. Another spike came that evening from an old bus idling at an intersection, about 35 $\mu g/m^3$ above baseline.

Rain returned on the fifth day. A brief spike was recorded again, shortly after it stopped, likely from resuspended dust. Humidity was above 85 % at the time so the reading may have been inflated. There was no way to confirm that. Only three alerts came through that entire day.

Fewer than twenty spikes were logged across the whole second week. Each one led to some kind of response, whether looking around, crossing the street, or making a note to avoid a spot. Scrolling through the history at the end of the day gave a short summary of the worst moments. That was very different from the hundreds of identical warnings in week one.

This was an informal case study, not a controlled experiment. One person, one city, two weeks.

6 Discussion

Fixed thresholds become noise almost immediately in a city where the baseline is always high. The two weeks mainly highlighted that. Adaptive alerts tied to the user's own recent exposure are rare enough that they actually get noticed, as they are more personalized. The ABSD algorithm handled this. It shifted with the baseline and only fired when something changed. The cooldown stopped the repeated burst that made week one so difficult.

The reset button was used more than expected, mostly after leaving air-conditioned rooms. Without the reset, the old indoor readings stay in the buffer and outdoor values trigger alerts constantly even when conditions have not changed relative to what is outside. A physical button for this turned out to be the right decision.

There are several limitations to note. The case study was based on a single user, in one searon only, summer, so the findings cannot be generalised. No humidity correction was applied and since the PMS7003 over-reads in humid air, so some of the rain-related spikes may have been inflated. The device needs a nearby phone to send BLE alerts. A dead phone battery means no alerts at all. The detection parameters were picked informally and cannot be changed by the user. The hardware had no proper enclosure either, so airflow to the sensor was not always consistent. So the viability of this approach in other conditions is still uncertain.

The parts are cheap and locally available. The whole thing costs around 4,500 BDT and the design can be copied without any special tools.

7 Conclusion

A portable PM2.5 monitor was constructed and carried for two weeks in Dhaka. It did not alert on a fixed limit, instead it alerted when a statistically significant spike occurred above the adaptive baseline. Hardware was an ESP32-C3, a PMS7003, a BME280 and a single 18650 cell on a small perfboard. On the other hand, feedback and spike history were handled by a React Native app and a Firebase backend.

One user had fewer than twenty alerts in a week using spike-detection mode. Each one was read and acted on as they were based on the user's immediate environment. Meanwhile, the Fixed-threshold mode produced hundreds that were ignored. The difference was noticeable from day two of the second week.

Larger trials across different seasons, humidity correction and user-adjustable parameters are the obvious next steps. The approach could also be tried in other heavily polluted cities where fixed thresholds have the same problem. Before any of that the system needs to be tested on more than one person. So whether these results hold more broadly is still an open question.

Furthermore, since the aim of the build was to be cost-effective and easy to construct, the hardware may not be the most accurate in terms of detection, but it is still usable for the stated purpose and functions perfectly for a day-to-day user.

References

[1] Brigida Alfano, Luigi Barretta, Antonio Del Giudice, Saverio De Vito, Girolamo Di Francia, Elena Esposito, Fabrizio Formisano, Ettore Massera, Maria Lucia Miglietta, and Tiziana Polichetti. 2020. A Review of Low-Cost Particulate Matter Sensors from the Developers' Perspectives. *Sensors* 20, 23 (2020), 6819. doi:10.3390/s20236819

[2] Vanessa E. Alvear-Puertas, Yadira A. Burbano-Prado, Paul D. Rosero-Montalvo, Pinar Tozun, Fabricio Marcillo, and Wilmar Hernandez. 2022. Smart and Portable Air-Quality Monitoring IoT Low-Cost Devices in Ibarra City, Ecuador. *Sensors* 22, 18 (2022), 7015. doi:10.3390/s22187015

[3] Marek Badura, Piotr Batog, Anetta Drzeniecka-Osiadacz, and Piotr Modzel. 2018. Evaluation of Low-Cost Sensors for Ambient PM2.5 Monitoring. *Journal of Sensors* 2018 (2018), 5096540. doi:10.1155/2018/5096540

[4] Janet G. Baseman, Debra Revere, Ian Painter, Mariko Toyoji, Hanne Thiede, and Jeffrey Duchin. 2013. Public Health Communications and Alert Fatigue. *BMC Health Services Research* 13 (2013), 295. doi:10.1186/1472-6963-13-295

[5] Anna Maria Becker, Heike Marquart, Torsten Masson, Carolin Helbig, and Uwe Schlink. 2021. Impacts of Personalized Sensor Feedback Regarding Exposure to Environmental Stressors. *Current Pollution Reports* 7 (2021), 579–593. doi:10.1007/s40726-021-00209-0

[6] Sara Bernasconi, Alessandra Angelucci, Stefano Canali, and Andrea Aliverti. 2024. Exploring air pollution awareness: insights from interviews with technology enthusiasts using wearable monitoring devices. *Frontiers in Public Health* 12 (2024), 1468662. doi:10.3389/fpubh.2024.1468662

[7] A. K. M. I. Haque, S. S. Islam, and M. S. Alam. 2020. A Low-Cost IoT-Based Air Quality Monitoring System for Developing Countries. In *2020 IEEE Region 10 Symposium (TENSYMP)*. 1554–1557. doi:10.1109/TENSYMP50017.2020.9230982

[8] Abhinav Mehrotra, Robert Hendley, and Mirco Musolesi. 2016. PrefMiner: Mining User's Preferences for Intelligent Mobile Notification Management. In *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp '16)*. 1223–1234. doi:10.1145/2971648.2971747

[9] Lidia Morawska, Phong K. Thai, Xiaoting Liu, Akwasi Asumadu-Sakyi, Godwin A. Ayoko, Alena Bartonova, Andrea Bedini, Fahe Chai, Bryce Christensen, Matthew Dunbabin, Jian Gao, Gayle S.W. Hagler, Rohan Jayaratne, Prashant Kumar, Alexis K.H. Lau, Peter K.K. Louie, Mandana Mazaheri, Zhi Ning, Nelson Motta, Ben Mullins, Md Mahmudur Rahman, Zoran Ristovski, Mohammad Shafiei, Dian Tjondronegoro, Dane Westerdahl, and Rohan Williams. 2018. Applications of low-cost sensing technologies for air quality monitoring and exposure assessment. *Environment International* 116 (2018), 286–299. doi:10.1016/j.envint.2018.04.018

[10] Vince Nguyen. 2024. myAQM: Interfacing Portable Air Quality Monitor with the Apple Watch – An In-the-Wild Usability Study. In *Pervasive Computing Technologies for Healthcare (PervasiveHealth 2023) (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Vol. 488)*. Springer, 339–363. doi:10.1007/978-3-031-59717-6_23

[11] Sergio Palomeque-Mangut, Félix Meléndez, Jaime Gómez-Suárez, Samuel Frutos-Puerto, Patricia Arroyo, Eduardo Pinilla-Gil, and Jesús Lozano. 2022. Wearable system for outdoor air quality monitoring in a WSN with cloud computing: Design, validation and deployment. *Chemosphere* 307 (2022), 135948. doi:10.1016/j.chemosphere.2022.135948

[12] Johanna Amalia Robinson, David Kocman, Milena Horvat, and Alena Bartonova. 2018. End-User Feedback on a Low-Cost Portable Air Quality Sensor System—Are We There Yet? *Sensors* 18, 11 (2018), 3768. doi:10.3390/s18113768

[13] Shailendra K. Sharma and Anil Kumar. 2020. IoT based Air Pollution Monitoring System using Blynk Server. In *2020 IEEE International Conference on Computing, Power and Communication Technologies (GUCON)*. 667–671. doi:10.1109/GUCON48875.2020.9231216

[14] Domenico Suriano. 2021. A Portable Air Quality Monitoring Unit and a Modular, Flexible Tool for On-Field Evaluation and Calibration of Low-Cost Gas Sensors. *HardwareX* 9 (2021), e00198. doi:10.1016/j.ohx.2021.e00198

[15] World Health Organization. 2021. *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva.